1. Problem Title & Source

Title: **Square Free Numbers**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

A **square-free number** is a number that is **not divisible by the square of any prime number**.

In other words:

- If a number contains a factor like $\mathbf{p^2}$ (where p is a prime), then it is **not square-free**.
- Otherwise, the number **is square-free**.

You are given a number **N**.

Your task is to determine whether **N is a square-free number**.

If the number is square-free, print:

YES

Otherwise print:

NO

3. Examples

Example 1

Input

10

Output

YES

Explanation

Prime factorization:

$10 = 2 \times 5$

There is **no squared prime factor**, so 10 is **square-free**.

Example 2

Input

12

Output

NO

Explanation

Prime factorization:

$12 = 2^2 \times 3$

Since $\mathbf{2^2}$ exists, the number is **not square-free**.

Example 3

Input

17

Output

YES

Explanation

17 is a prime number

It has **no squared factors**, so it is square-free.

4. Constraints

1 ≤ N ≤ 10^6

Observations

- We only need to check divisibility by **square numbers**
- Only check primes up to **√N**

5. Core Concept (Pattern)

This problem involves:

Prime factorization

Square factor detection

Key idea:

If N % (i*i) == 0 → not square-free

6. Thought Process

Step 1 — Identify Square Factors

If any number i^2 divides N, then:

N is not square-free

Example

12 % (2^2) = 12 % 4 = 0

So **12 is not square-free**.

Step 2 — Limit the Search

We only need to check values of i up to:

√N

Because if $i^2 > N$, it cannot divide N.

7. Visualization

Example

N = 18

Check squares:

i	i^2	Divides 18?
2	4	No
3	9	Yes

Since

18 % 9 = 0

Result

NOT square-free

8. Pseudocode

```
read N

for i from 2 to √N

   if N % (i*i) == 0
      print NO
      stop

print YES
```

9. Code Implementation

Python

```
import math

n = int(input())

for i in range(2, int(math.sqrt(n)) + 1):

   if n % (i*i) == 0:
      print("NO")
      break
else:
   print("YES")
```

Java

```
import java.util.*;

class Main {

   public static void main(String[] args){

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      for(int i=2;i*i<=n;i++){

         if(n % (i*i) == 0){

            System.out.println("NO");
            return;
         }
      }

      System.out.println("YES");
   }
}
```

10. Time & Space Complexity

Time Complexity

O(√N)

Because we only check numbers up to √N.

Space Complexity

O(1)

Only constant variables used.

11. Common Mistakes

1. Checking N % i == 0 instead of N % (i*i) == 0
2. Looping until **N instead of √N**
3. Incorrect square calculation
4. Forgetting early termination

12. Detailed Dry Run

Input

N = 20

Check squares

i	i^2	Divides ?
2	4	Yes

Since

20 % 4 = 0

Output

NO

13. Real-Life Applications

Square-free numbers appear in:

- Cryptography
- Number theory
- Mathematical modeling
- Prime factor analysis

14. Common Traps

1. Confusing prime check with square-free check
2. Checking squares incorrectly
3. Missing i*i logic
4. Running loop unnecessarily long

15. Related Problems

Number theory problems:

LC 204 Count Primes

LC 2520 Count Digits That Divide Number

LC 507 Perfect Number

16. Alternate Approaches

Approach	Complexity	Notes
Prime factorization	$O(\sqrt{N})$	works
Square divisor check	$O(\sqrt{N})$	simpler

17. Why This Solution Works

A number is **square-free only if it has no squared prime factor**.

By checking divisibility by i^2, we ensure that **no squared factor divides the number**, which confirms the condition.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Count **square-free numbers in range**
2. Find **largest square-free number ≤ N**
3. Remove square factors from number
4. Check if number is **perfect square**